Python Case Studies 22–30

Simple flowcharts • 4-line Python code • sample test cases and output

How to read this PDF: Each flowchart follows the same easy pattern as your notebook example: Start → Input → Process/Check → Display → Stop. The rules and amounts are sample assumptions for learning; use your teacher's exact values if provided.

Flowchart symbols

Oval = Start/Stop Slanted box = Input/Output or processing step Arrow = direction of flow

Case Study 22: Traffic Violation Penalty Calculator

Problem statement: Calculate a sample penalty based on the type of traffic violation.

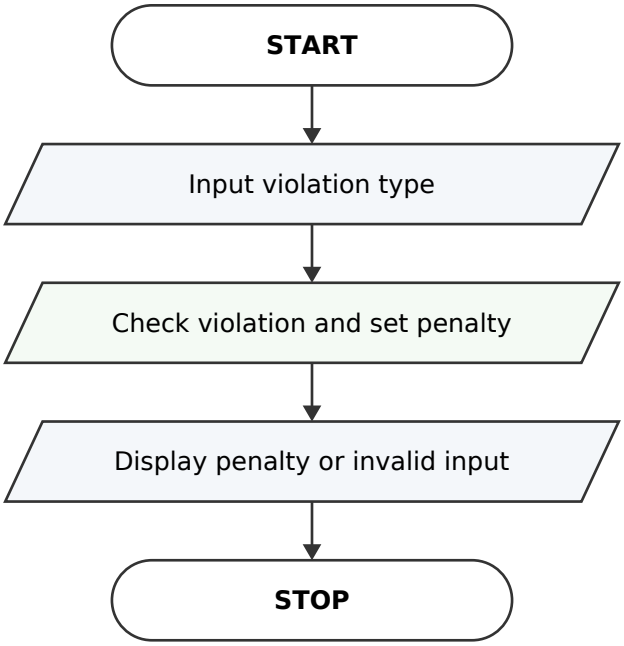
Objective: Accept a violation type and display its penalty.

Assumption: Illustrative penalties: helmet = ₹500, signal = ₹1000, speeding = ₹1500. Other inputs are invalid.

Algorithm

1. Start
2. Input violation type
3. Check violation and set penalty
4. Display penalty or invalid input
5. Stop

Flowchart



Case Study 22: Python Code and Output

Traffic Violation Penalty Calculator

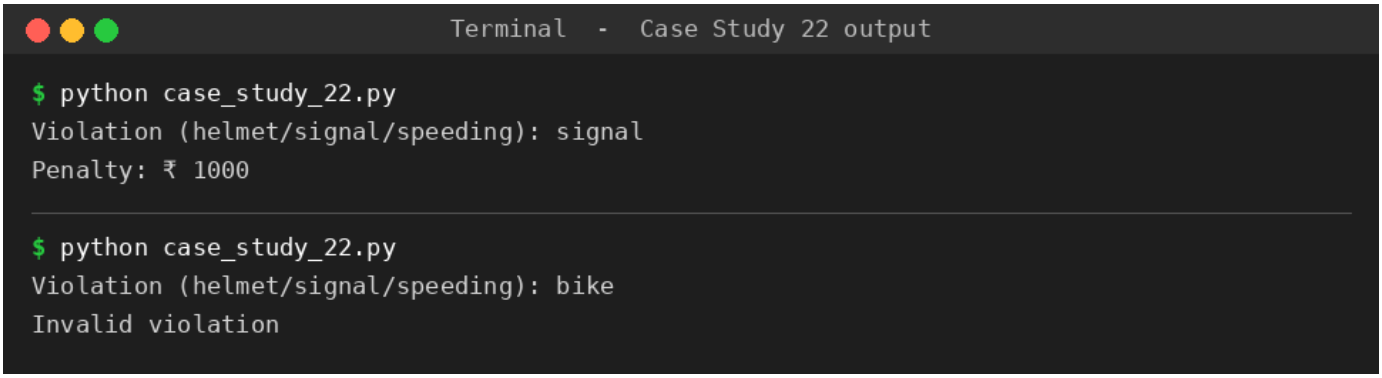
Python code (4 lines)

```python
v = input("Violation (helmet/signal/speeding): ").lower()
fees = {"helmet": 500, "signal": 1000, "speeding": 1500}
print("Penalty: ₹", fees[v]) if v in fees else print("Invalid violation")
# End
```

Sample test case

Input: **signal**

Sample output (screenshot)



*Test 1 is the sample test case. Test 2 is an extra case that checks the invalid / alternative path (input: **bike**).*

Conclusion

This program shows how a dictionary can link each violation type to its penalty, so a long if-elif chain is not needed. The membership check (**in**) reports unknown violations as invalid instead of crashing, and **.lower()** makes the input case-insensitive. Test 1 (signal) gives a penalty of ₹ 1000, and Test 2 (bike) is correctly rejected.

Case Study 23: Laboratory Chemical Inventory Validation

Problem statement: Check whether a chemical quantity entered by the user is valid.

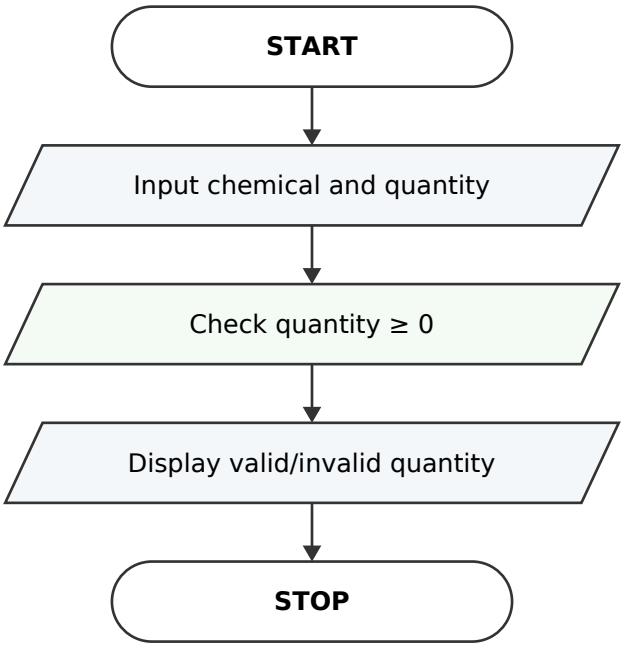
Objective: Accept a chemical name and quantity; reject negative quantities.

Assumption: Quantity must be zero or greater.

Algorithm

1. Start
2. Input chemical and quantity
3. Check quantity ≥ 0
4. Display valid/invalid quantity
5. Stop

Flowchart



Case Study 23: Python Code and Output

Laboratory Chemical Inventory Validation

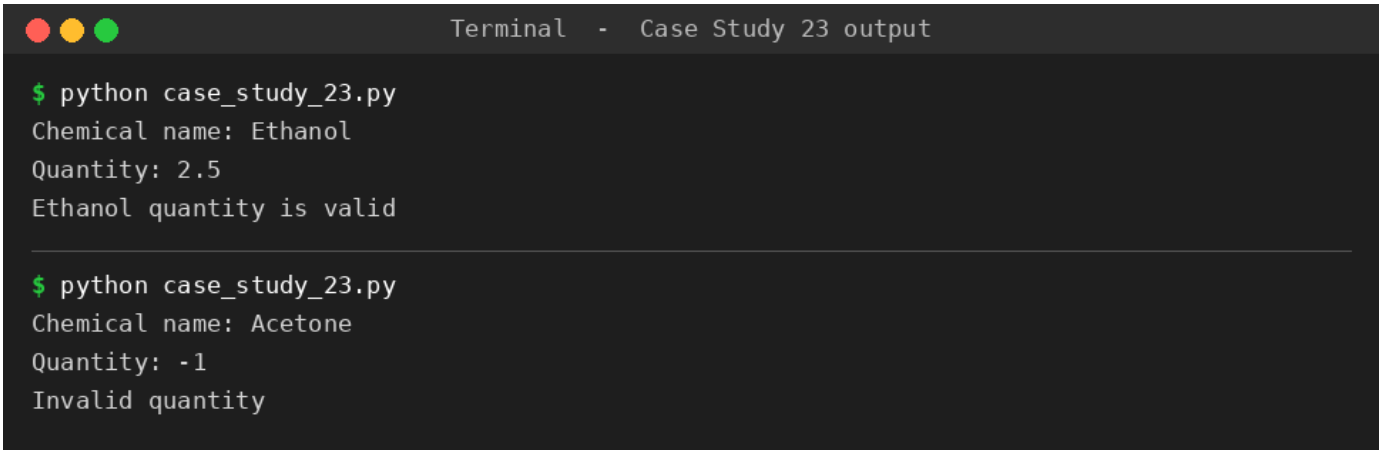
Python code (4 lines)

```
1  chemical = input("Chemical name: ")
2  qty = float(input("Quantity: "))
3  print(chemical, "quantity is valid") if qty >= 0 else print("Invalid quantity")
4  # End
```

Sample test case

Input: **Ethanol, 2.5**

Sample output (screenshot)



*Test 1 is the sample test case. Test 2 is an extra case that checks the invalid / alternative path (input: **Acetone, -1**).*

Conclusion

This program validates a stock quantity with a single comparison (**qty >= 0**). Converting the input with **float()** allows decimal quantities such as 2.5, and negative values are rejected as invalid. As a possible extension, a try/except block could handle non-numeric input.

Case Study 24: Student Timetable Conflict Detection

Problem statement: Check whether two classes are scheduled in the same time slot.

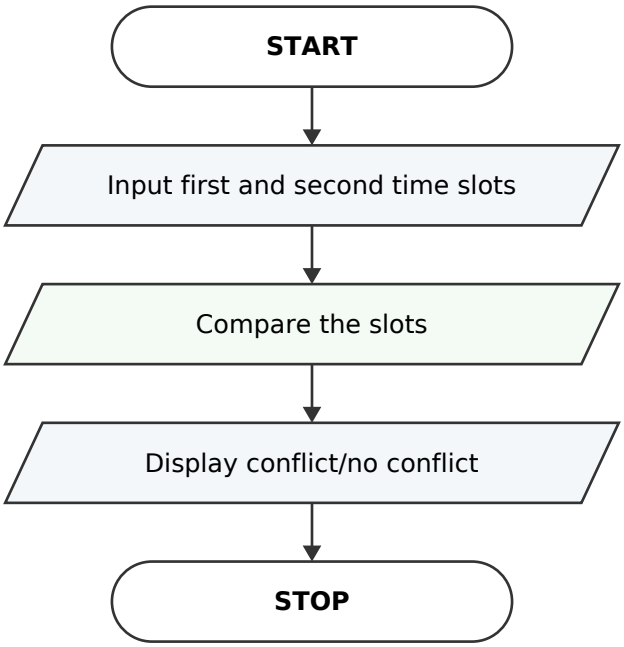
Objective: Compare two entered time slots and report a conflict.

Assumption: Matching time-slot text means a conflict.

Algorithm

1. Start
2. Input first and second time slots
3. Compare the slots
4. Display conflict/no conflict
5. Stop

Flowchart



Case Study 24: Python Code and Output

Student Timetable Conflict Detection

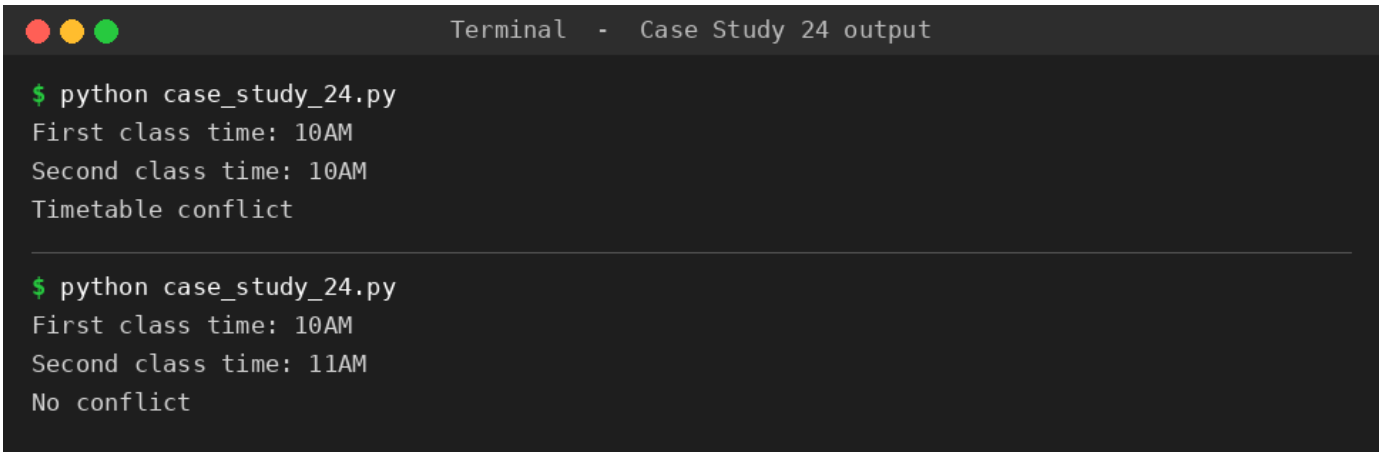
Python code (4 lines)

```
a = input("First class time: ")
b = input("Second class time: ")
print("Timetable conflict") if a == b else print("No conflict")
# End
```

Sample test case

Input: **10AM, 10AM**

Sample output (screenshot)



*Test 1 is the sample test case. Test 2 is an extra case that checks the invalid / alternative path (input: **10AM, 11AM**).*

Conclusion

This program detects a clash by comparing two time-slot strings with the equality operator (==). Identical slots give a conflict and different slots give no conflict. It only matches exact text, so a real timetable system would compare start and end times instead.

Case Study 25: Courier Charge Calculator

Problem statement: Calculate a simple courier charge from the parcel weight.

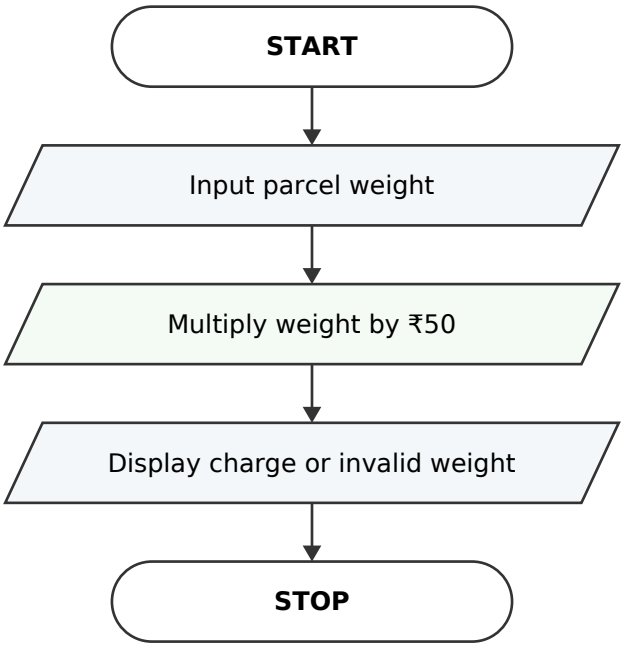
Objective: Display the charge using the entered weight.

Assumption: Sample rate is ₹50 per kg; weight must be positive.

Algorithm

1. Start
2. Input parcel weight
3. Multiply weight by ₹50
4. Display charge or invalid weight
5. Stop

Flowchart



Case Study 25: Python Code and Output

Courier Charge Calculator

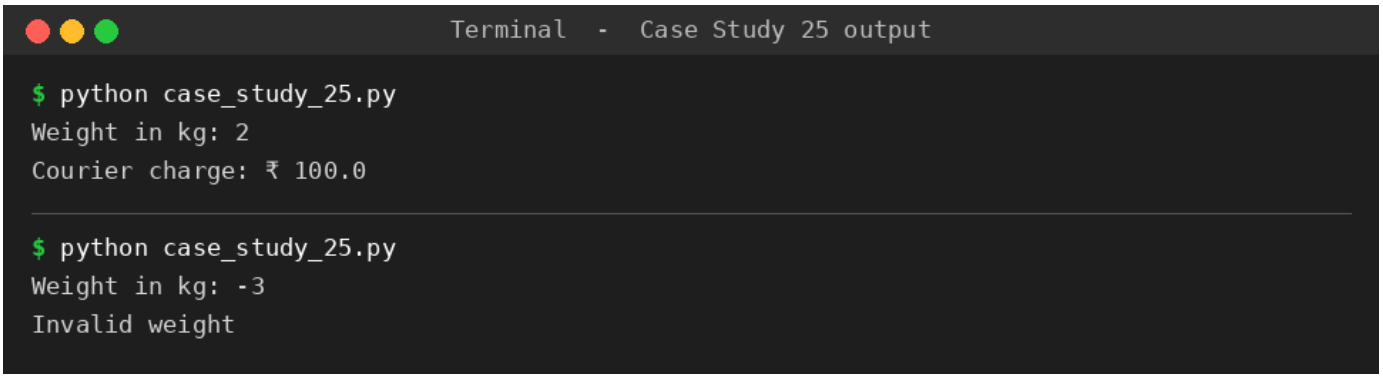
Python code (4 lines)

```
weight = float(input("Weight in kg: "))
rate = 50
print("Courier charge: ₹", weight * rate) if weight > 0 else print("Invalid weight")
# End
```

Sample test case

Input: **2**

Sample output (screenshot)



*Test 1 is the sample test case. Test 2 is an extra case that checks the invalid / alternative path (input: **-3**).*

Conclusion

This program calculates the charge by multiplying the parcel weight by a fixed rate of ₹ 50 per kg. The condition **weight > 0** prevents zero or negative weights from producing a meaningless charge. A weight of 2 kg gives ₹ 100.0.

Case Study 26: Progressive Water Tariff Analysis

Problem statement: Calculate a sample water bill based on units consumed.

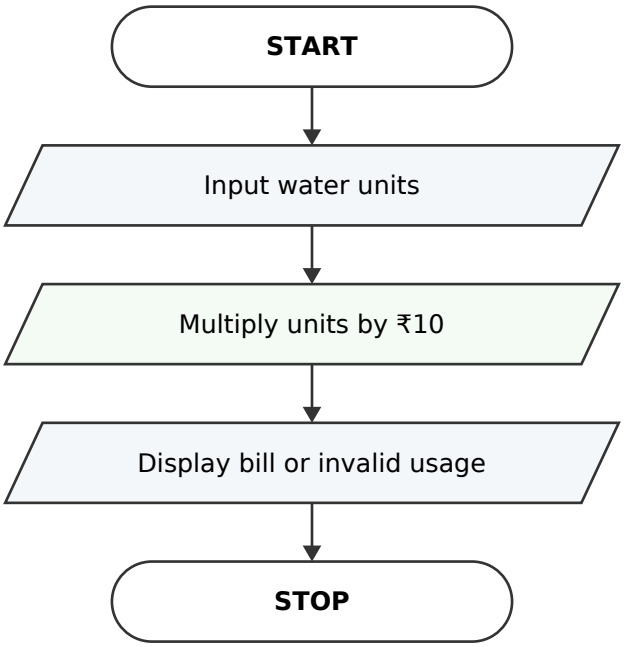
Objective: Apply a simple rate to the entered water usage.

Assumption: Sample rate is ₹10 per unit; usage must not be negative.

Algorithm

1. Start
2. Input water units
3. Multiply units by ₹10
4. Display bill or invalid usage
5. Stop

Flowchart



Case Study 26: Python Code and Output

Progressive Water Tariff Analysis

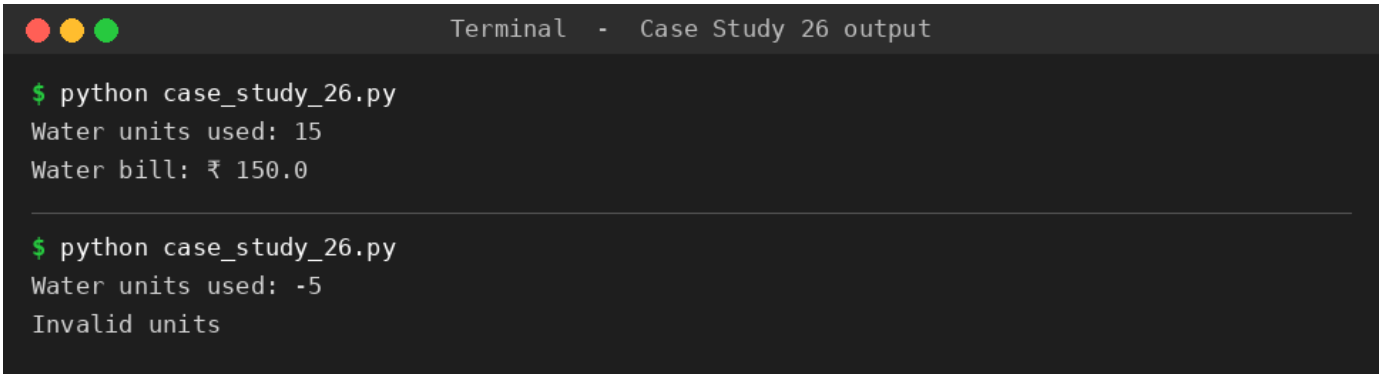
Python code (4 lines)

```
1  units = float(input("Water units used: "))
2  rate = 10
3  print("Water bill: ₹", units * rate) if units >= 0 else print("Invalid units")
4  # End
```

Sample test case

Input: **15**

Sample output (screenshot)



*Test 1 is the sample test case. Test 2 is an extra case that checks the invalid / alternative path (input: **-5**).*

Conclusion

This program applies a single sample rate of ₹ 10 per unit to the water used and rejects negative usage. A usage of 15 units gives a bill of ₹ 150.0. A real progressive tariff would use different rates for different slabs of usage, which can be added later using if-elif conditions.

Case Study 27: Examination Seating Generator

Problem statement: Generate seat numbers for a given number of students.

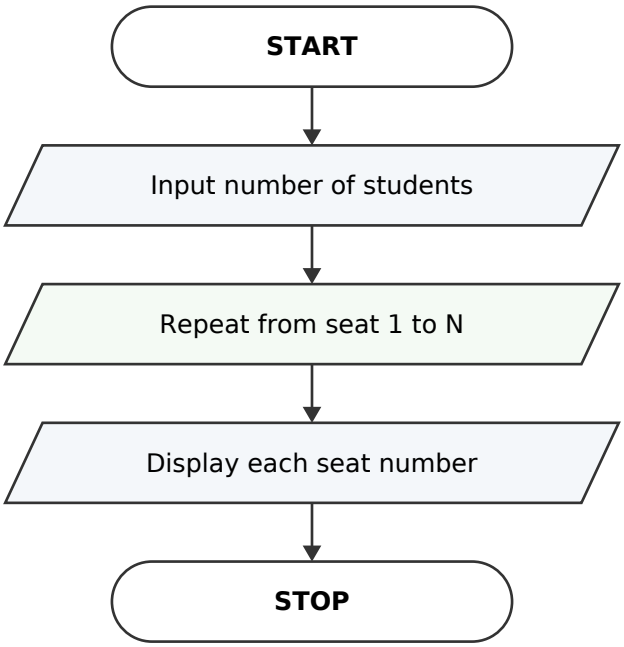
Objective: Print numbered seats from 1 up to the student count.

Assumption: The student count must be a positive whole number.

Algorithm

1. Start
2. Input number of students
3. Repeat from seat 1 to N
4. Display each seat number
5. Stop

Flowchart



Case Study 27: Python Code and Output

Examination Seating Generator

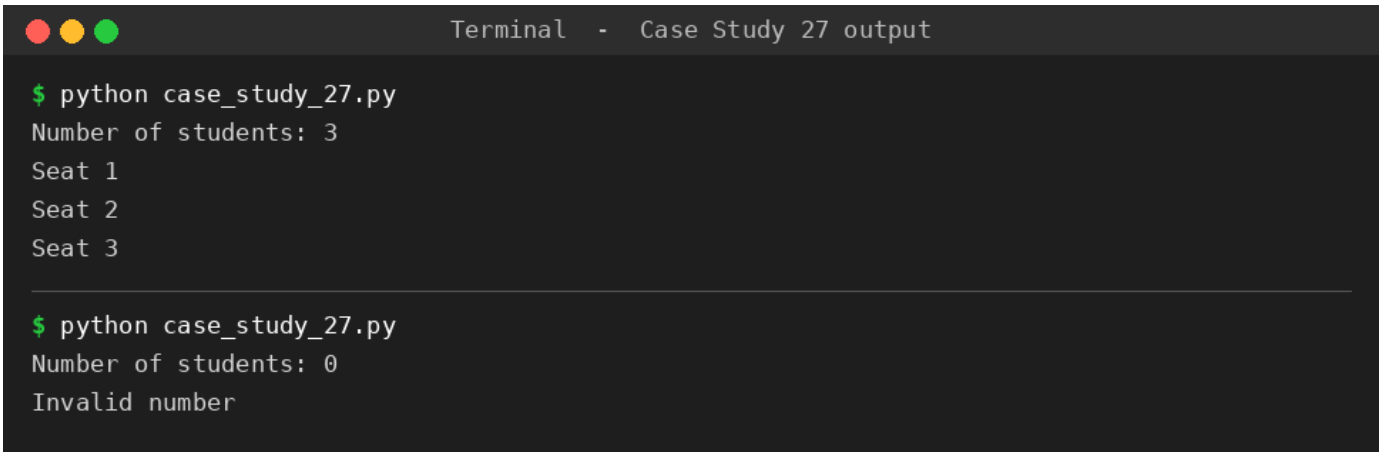
Python code (4 lines)

```
1 n = int(input("Number of students: "))
2 if n > 0:
3     [print("Seat", i) for i in range(1, n + 1)]
4 else: print("Invalid number")
```

Sample test case

Input: **3**

Sample output (screenshot)



*Test 1 is the sample test case. Test 2 is an extra case that checks the invalid / alternative path (input: **0**).*

Conclusion

This program uses **range(1, n + 1)** to generate seat numbers from 1 to the number of students, so the loop includes the last seat. The check **n > 0** rejects zero or negative counts. For 3 students it prints Seat 1, Seat 2 and Seat 3.

Case Study 28: Repeated Transaction Detection

Problem statement: Check whether a transaction ID has already been recorded in a small list.

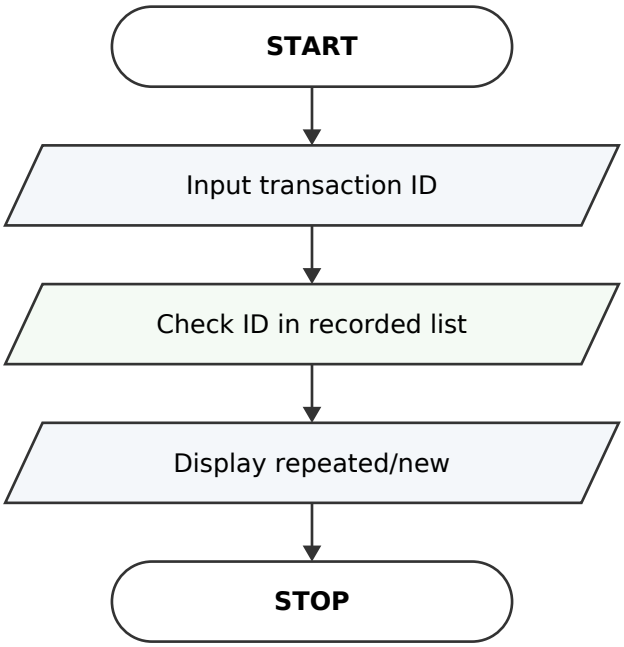
Objective: Report whether the entered transaction ID is repeated.

Assumption: The example list contains IDs T101 and T102.

Algorithm

1. Start
2. Input transaction ID
3. Check ID in recorded list
4. Display repeated/new
5. Stop

Flowchart



Case Study 28: Python Code and Output

Repeated Transaction Detection

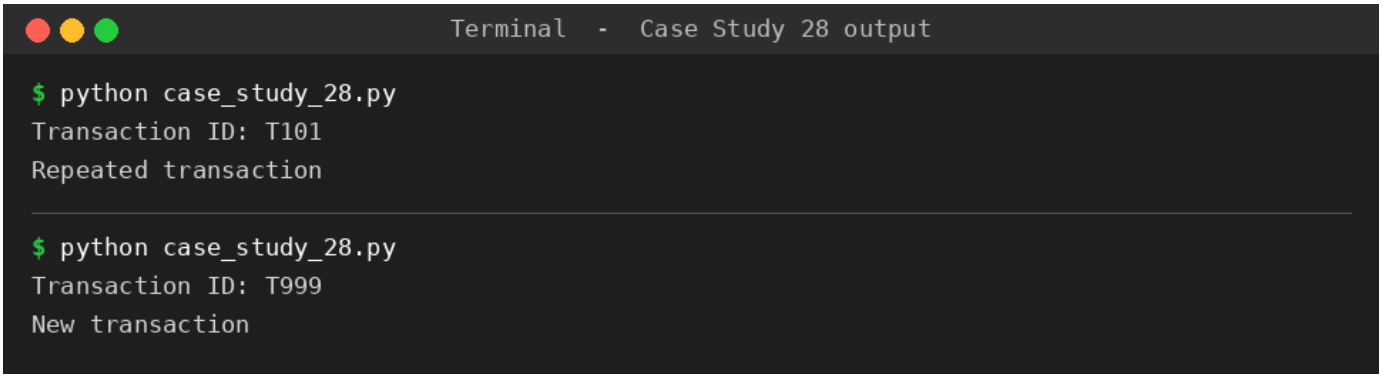
Python code (4 lines)

```
recorded = ["T101", "T102"]
tid = input("Transaction ID: ")
print("Repeated transaction") if tid in recorded else print("New transaction")
# End
```

Sample test case

Input: **T101**

Sample output (screenshot)



*Test 1 is the sample test case. Test 2 is an extra case that checks the invalid / alternative path (input: **T999**).*

Conclusion

This program checks whether an ID already exists in a list using the **in** operator. T101 is found in the recorded list, so it is reported as repeated, while T999 is reported as new. In a real system, new IDs would also be added to the list or a database so later repeats can be caught.

Case Study 29: Password Policy Validation

Problem statement: Check whether a password contains at least eight characters.

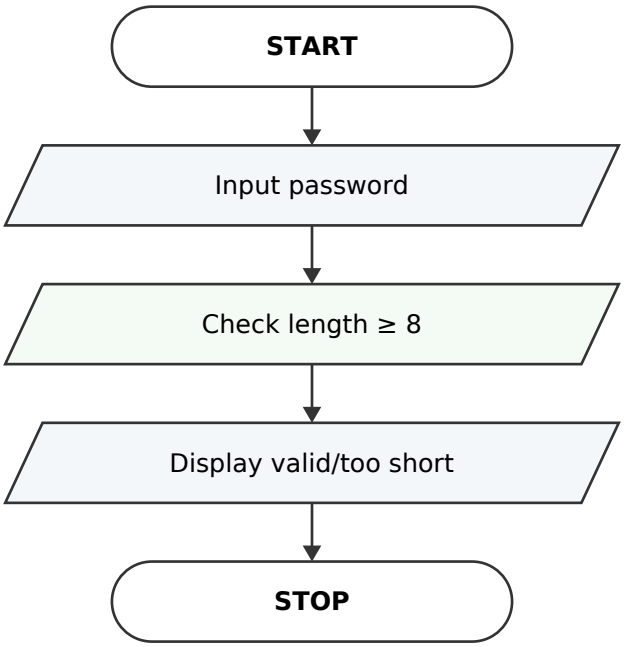
Objective: Display whether the entered password meets the sample length rule.

Assumption: For this simplified exercise, length ≥ 8 is the only rule checked.

Algorithm

1. Start
2. Input password
3. Check length ≥ 8
4. Display valid/too short
5. Stop

Flowchart



Case Study 29: Python Code and Output

Password Policy Validation

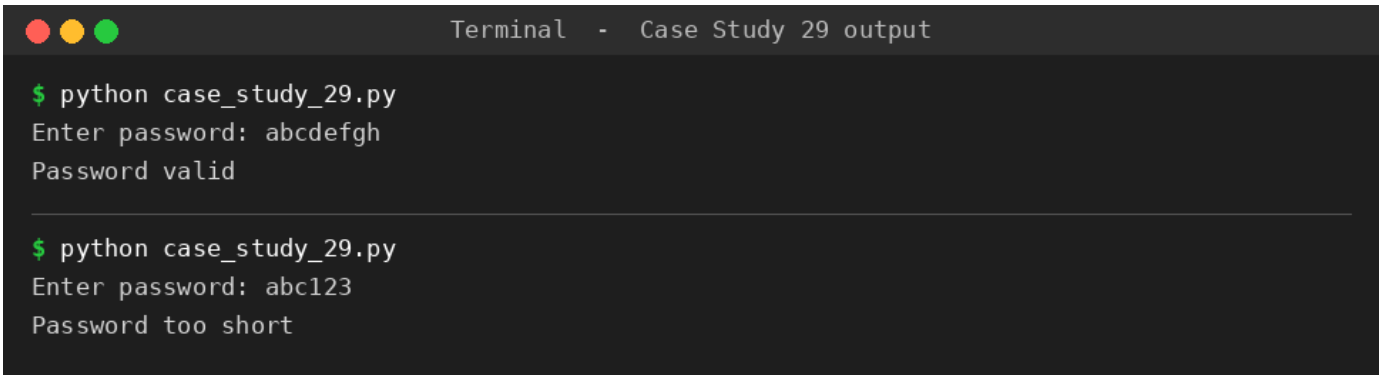
Python code (4 lines)

```python
password = input("Enter password: ")
valid = len(password) >= 8
print("Password valid") if valid else print("Password too short")
# End
```

Sample test case

Input: **abcdefgh**

Sample output (screenshot)



*Test 1 is the sample test case. Test 2 is an extra case that checks the invalid / alternative path (input: **abc123**).*

Conclusion

This program uses **len()** to count the characters and accepts a password only if it has at least 8. The 8-character password abcdefgh is valid and the 6-character abc123 is too short. Length is the only rule checked here, and a stronger policy would also require digits, upper-case letters and symbols.

Case Study 30: Student Email and Identifier Validation

Problem statement: Check whether an entered student email contains the @ symbol.

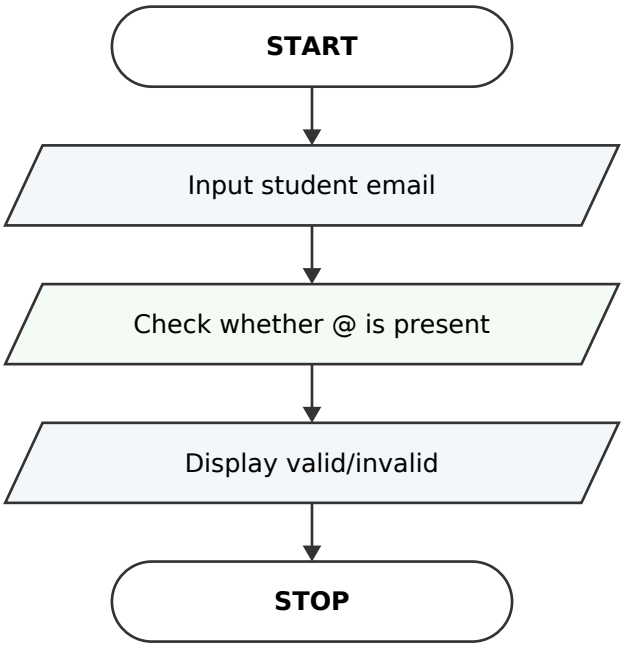
Objective: Perform a simple email-format check.

Assumption: This beginner example checks only for @; it does not fully validate email addresses.

Algorithm

1. Start
2. Input student email
3. Check whether @ is present
4. Display valid/invalid
5. Stop

Flowchart



Case Study 30: Python Code and Output

Student Email and Identifier Validation

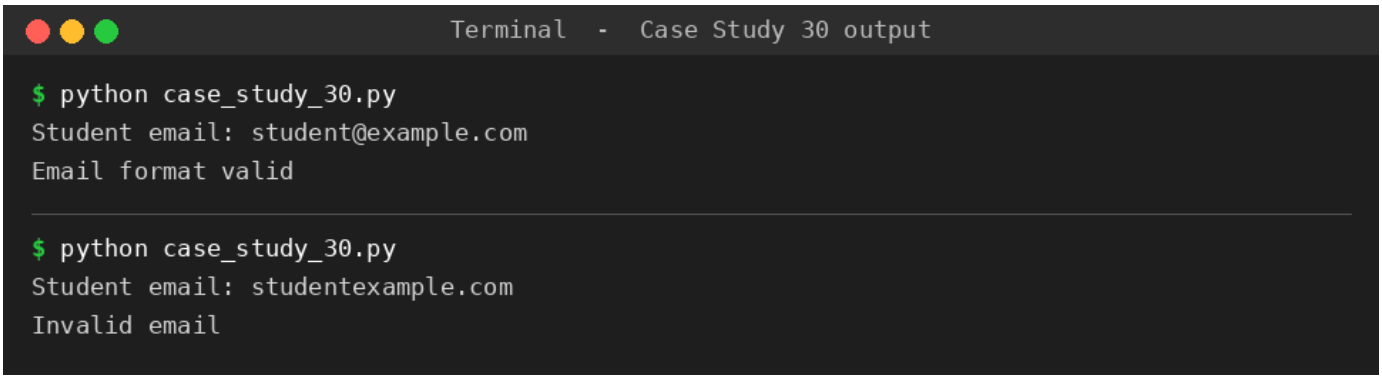
Python code (4 lines)

```
email = input("Student email: ")
valid = "@" in email
print("Email format valid") if valid else print("Invalid email")
# End
```

Sample test case

Input: **student@example.com**

Sample output (screenshot)



*Test 1 is the sample test case. Test 2 is an extra case that checks the invalid / alternative path (input: **studentexample.com**).*

Conclusion

This program performs a simple email check by testing whether the @ symbol is present in the text. An address with @ passes and one without it is rejected. This is only a beginner-level check, and full validation would also test the domain part and the characters used.